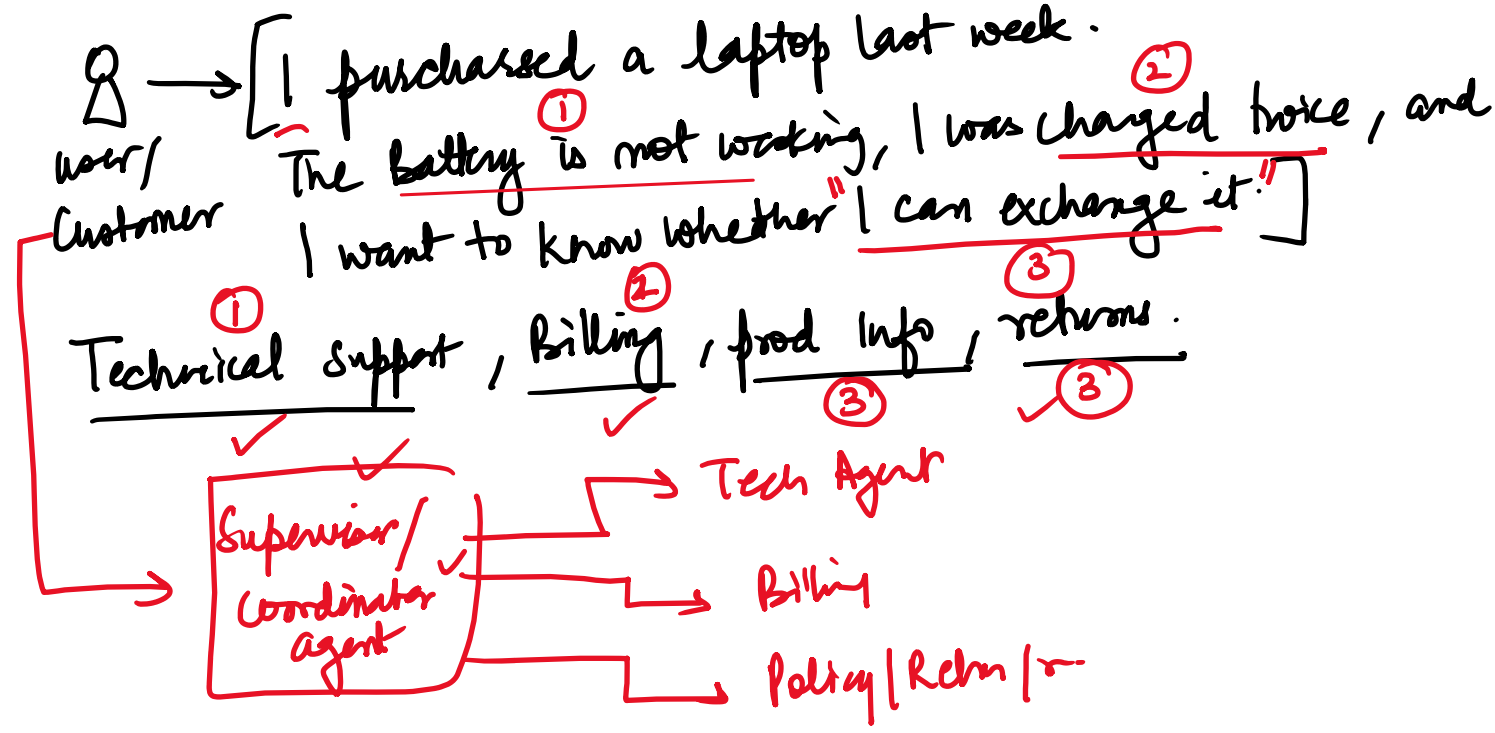
AWS AI/ML Stack

- Amazon Bedrock ⇒ 270+ FMs/LLMs → Development
- Amazon Bedrock AgentCore ? Deployment + Scale
- open-source frameworks - Langchain, STRANDS



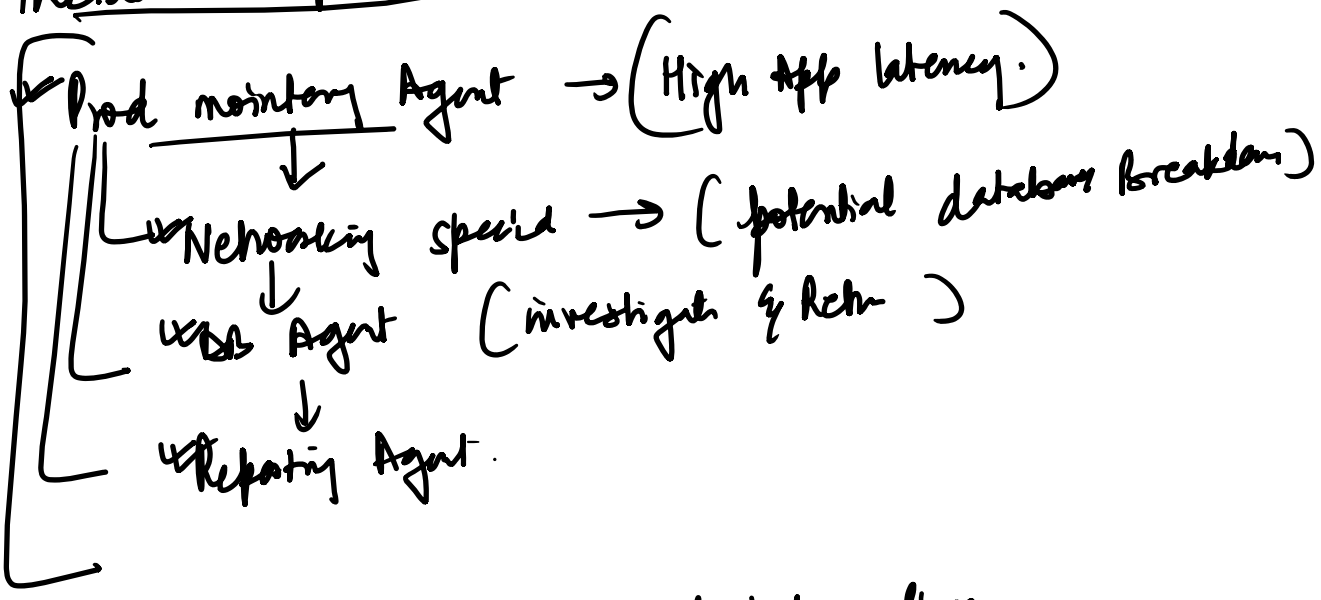
Amazon Bedrock AgentCore - Runtime

① Managed Hosting & Scaling

Calibration/Validation layer
- Evidence
- Consistency checks
- Confidence

Incident Response

Incident Response



Context Info = The current interaction

Persistent memory = Info that remains available for later interactions.

AgentCore Memory

1) Memory Resource = Managed Container
2) Actor = the entity to which events belong.
3) Session group = a particular interaction
4) Namespace = long-term memory records

These identifies determine How CONTEXT is isolated & retrieved.

Create_memory_session = a session for memory operate.

actor-id = memory events

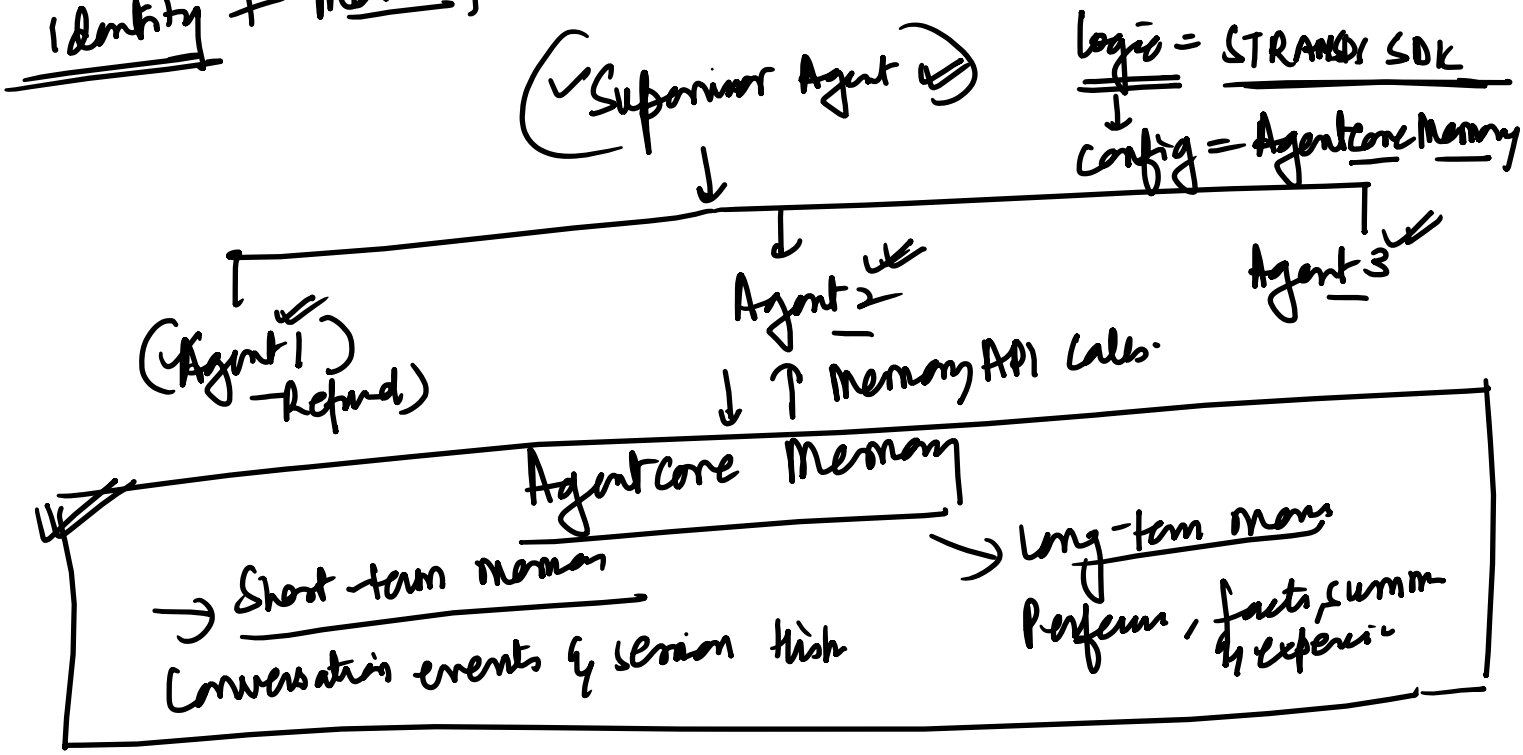
session-id = Interaction session

Short_term_Hooks = Connects agent lifecycle operate to Short-term memory

Short_term_Hooks = Context ... memory

Agent = Sys Prompt + agent(Qwen)

Identity + Memory



Shared Memory / Isolated memory

Design	Configuration	Behavior
Shared session memory	→ Same memory resource, "short-term" actorId & sessionId	→ Agents can retrieve the same session events if authorized (identity)
Agent-specific memory	→ actorId & sessionId → Separate actor/session combinations	→ each agent maintains independent working context
Shared Long-term memory	→ same accessible "namespace"	→ Agents can reuse stored insights
Private Long-term memory	→ Separate "namespaces" with IAM restrictions	→ Limit access to authorized agents

... "... configuration"

memory

Example "memory configuration"

Shared workflow context:

memoryresource → FinAdvisorMemory
ActorID — custom123
SessionID — workflow-456

Long-term namespaces

Shared facts : /customer/customer-123/shared/
Research agent's knowledge = /customer/customer-123/research

Q* App what to capture ? → Raw conv, Tool results, data, agent-generated summ
* memory (AgentCore)
Q. what to remember → Imp facts, preferences, decisions, exp, extracted through memory
Q* App what to share ? Which parts of that memory another agent needs for its current task.
↳ Imp decisions.

AgentCore provides Built-in memory

Strategy	What gets remembered	Example
✓ Semantic	- Facts & useful knowledge	- Customer has a credit limit of 5K
✓ Summarization	- Imp conversation topics, activities, decisions	- Customer requested for a loan eligibility Assessment
✓ user preference	- user preferences, specific choices	→ email communication.

- User preferences
  - Choices
- Significant experiences & outcomes
- Semantic

✓ Episodic

Practical example: What should agents actually share?

3 agents procesing a loan application

Q → "I want to apply for a $10K loan. My salary is $5K. I have an existing car loan with an EMI $1.5K. I prefer 5 year repayment period.

Customer Service Agent → Strands SDK, Agentcore SDK
(Receive the conversation & record relevant facts)

Agentcore Memory (Placeholder)

Context
- Loan Amount =
- Income =
- Existing EMI =
- Prefer tenure =

Risk Agent
Income, EMI, loan Amount relevent verified Risk data

Recommendation Agent
Preferred tenure, loan Amt, Approved affordable amount

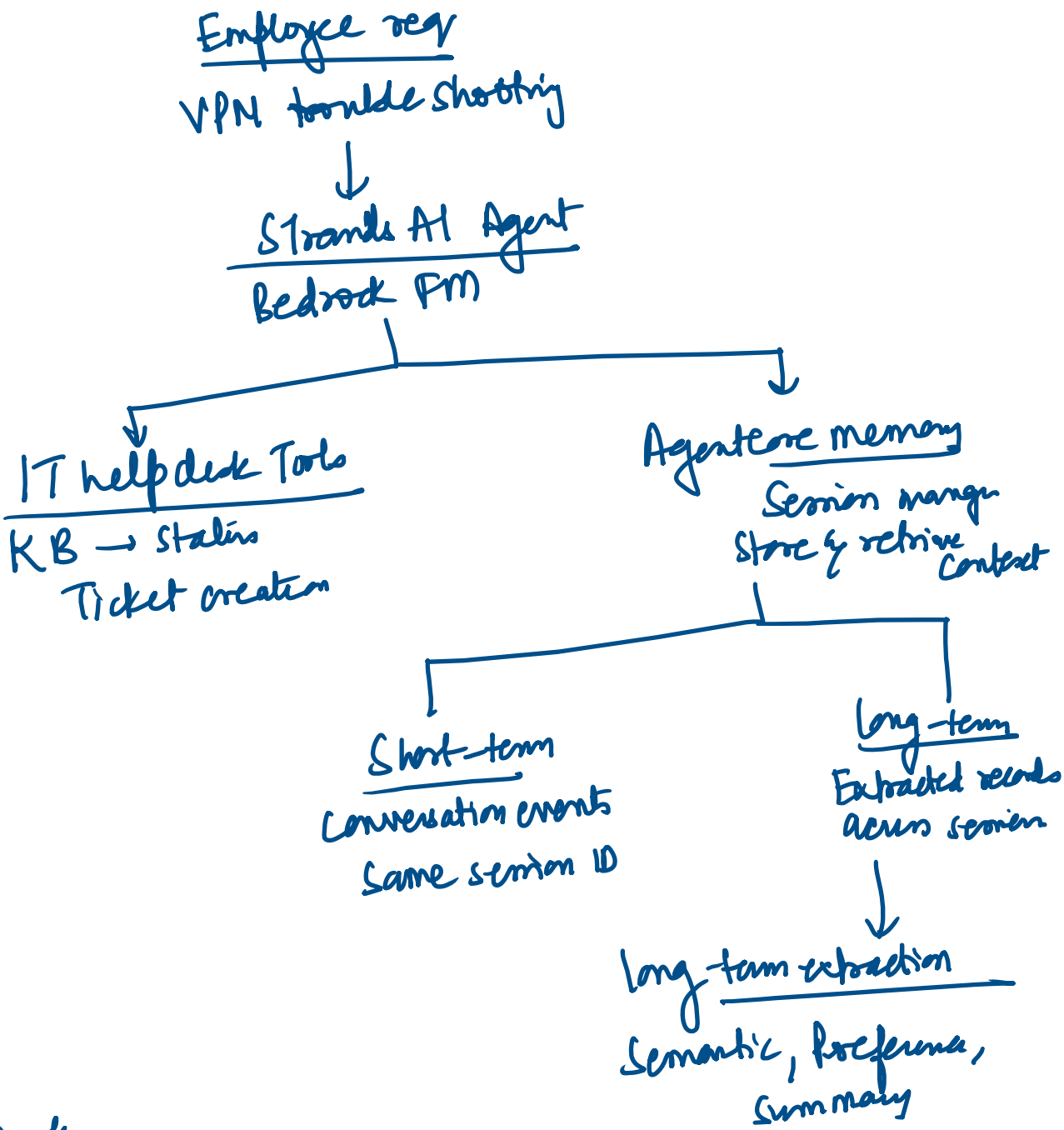
Demo: Context-aware Agent with Bedrock Agentcore memory.

(1) Create a Agentcore Memory resource configured with Semantic, User preference, Summary strategies.

(2) Integrate persistent memory into STRANDS agent & long-term retrieval to enable cross-

(2) Integrate persistent memory configure long-term retrieval to enable cross-session recall for a returning user.

(3) Inspect stored memories in the AgentCore Memory console.



NoteBook

(1) Baseline IT Helpdesk Agent → Agent, model, sys prompt, tools

(2) Agentcore Memory resource → Semantic, preference, summary

(2) AgentCore Memory resource → Semantic, preference, summary

(3) Memory enabled strands Agents
↳ Session Manager, actor ID, session ID, short term memory

(4) Two sessions with the same employee = long-term extraction

Monday:

Secure AI Agent Access with AgentCore Identity

(1) Deploy an agent to Amazon Bedrock AgentCore Runtime with AgentCore Identity inbound Auth.

(2) Amazon Cognito user pool, get a JWT, decode it to see the claims AgentCore Identity checks

(3) Confirm AgentCore Identity reject callers without a valid JWT & lets valid ones through.

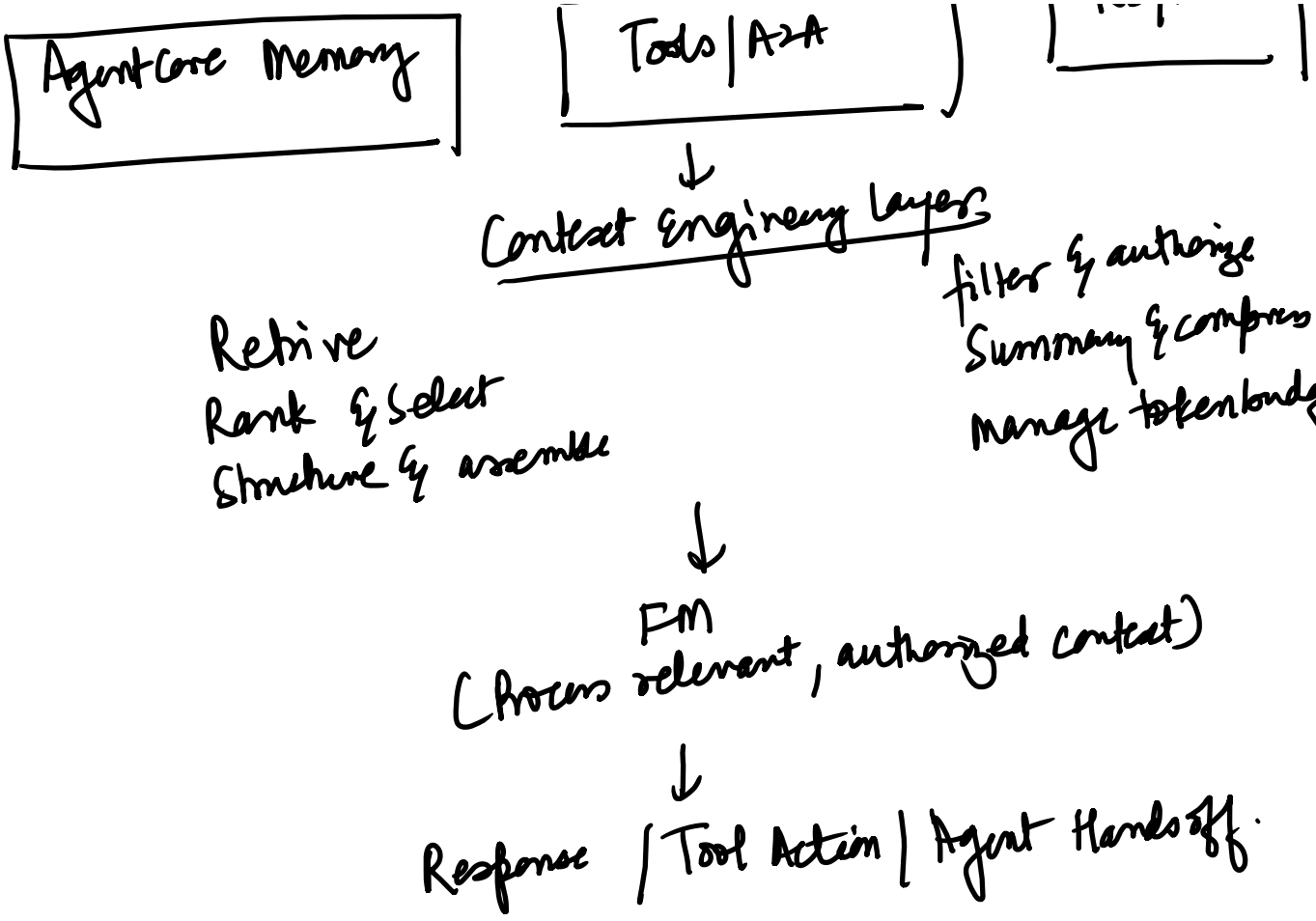
Context engineering in Generative AI & Multi-Agent Systems

Process of designing, selecting, organizing, managing the info to the AI Model / Agent so that it can produce accurate, relevant & reliable results.

What info should each agent receive, at what time, in what format, for what purpose?

Agent / Current Task

AgentCore Memory	Tools / A2A	KB / RAGs


AgentCore Memory
Tools | A2A
Context Engineering Layers
Retrive
Rank & Select
Structure & assemble
filter & authorize
Summary & compress
manage tokenbudget
FM
(Process relevant, authorized context)
Response | Tool Action | Agent Handoff.

Day 2:

Day 1 Recap.

"Limitations" of Single-agent:

1. Cognitive load limitations
2. Context window exhaustion
3. Domain specialization

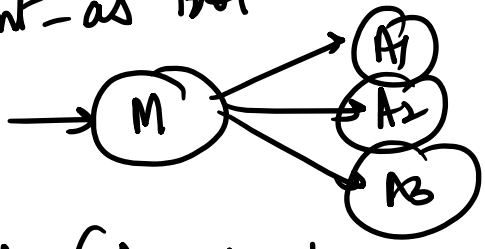
Benefits of multi-agent ⇒ Also, uses Microservices BP:

1. Reusability
2. isolation & maintainability
3. fault tolerance & Resilience
4. Scalability & performance

Q How do multiple agents communicate?

THREE

1. Framework-Layer. = Langchain / STRANDS
2. Agent-as tool!



3. A2A (Agent-to-agent) Protocol

↳ components of A2A

- A2A Client, A2A Server, Agent Card, Task, Artifact

Task, Artifact

"MCP (Model Context Protocol)" — A2A → Communication agents

↳ Expose "Tools" & Capabilities of agents

"AgentCore Runtime" = Managed Hosting for agents

frameworks (STRANDS, langchain, crewAI) → How the agents coordinate

MEMORY & STATE Management

3 Types of Information agents may need to RETAIN

(1) Contextual Info: what is happening in the current conversation?

(2) user preferences: what preferences should remain available?

(3) Retained Knowledge: what useful info should keep across the sessions?

↳ AgentCore Memory

↳ SHORT-term = Conversational events & immediate interaction context

↳ long-term = useful info — preferences, imp facts, etc

Memory integration (Identifiers)

1) memory-id = memory resource

2) session-id = interaction session

3) actor-id = Actor associated with memory events.

2) "session_id" = [illegible]
3) "actor_id" = Actor associated with memory events.

STRANDS: invocation_state
↳ Relevant execution info = user context, session identifier, app config

Context engineering - Generative AI & Multi-Agent Systems

↳ Process of designing, selecting, organizing & managing the "information" provided to AI Model (LLM) so that it can produce Accurate, Relevant & Reliable Results.

"Amazon Bedrock 'AgentCore' "Memory"

what info should each agent receive, at what time, in what format, for what purpose?

"Content window"

↳ Info available to the model for the current inference request.

① Sys Instructions = Role, scope, policies
② User request = Current objective

(2) User request = Current objective
(3) Conversation History = Relevant prior interactions
(4) Memory = Save facts & Prefer (Long term)

Day 2 - Agenda

(1) Security & Compliance Implementation

Demo: Secure AI Agent Access with AgentCore Identity

★ Demo - Integrate Agents with MCP Servers & Tools using AgentCore Gateway.

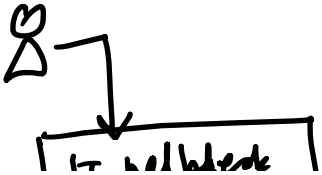
(2) Production Monitoring, Observability & Evaluation

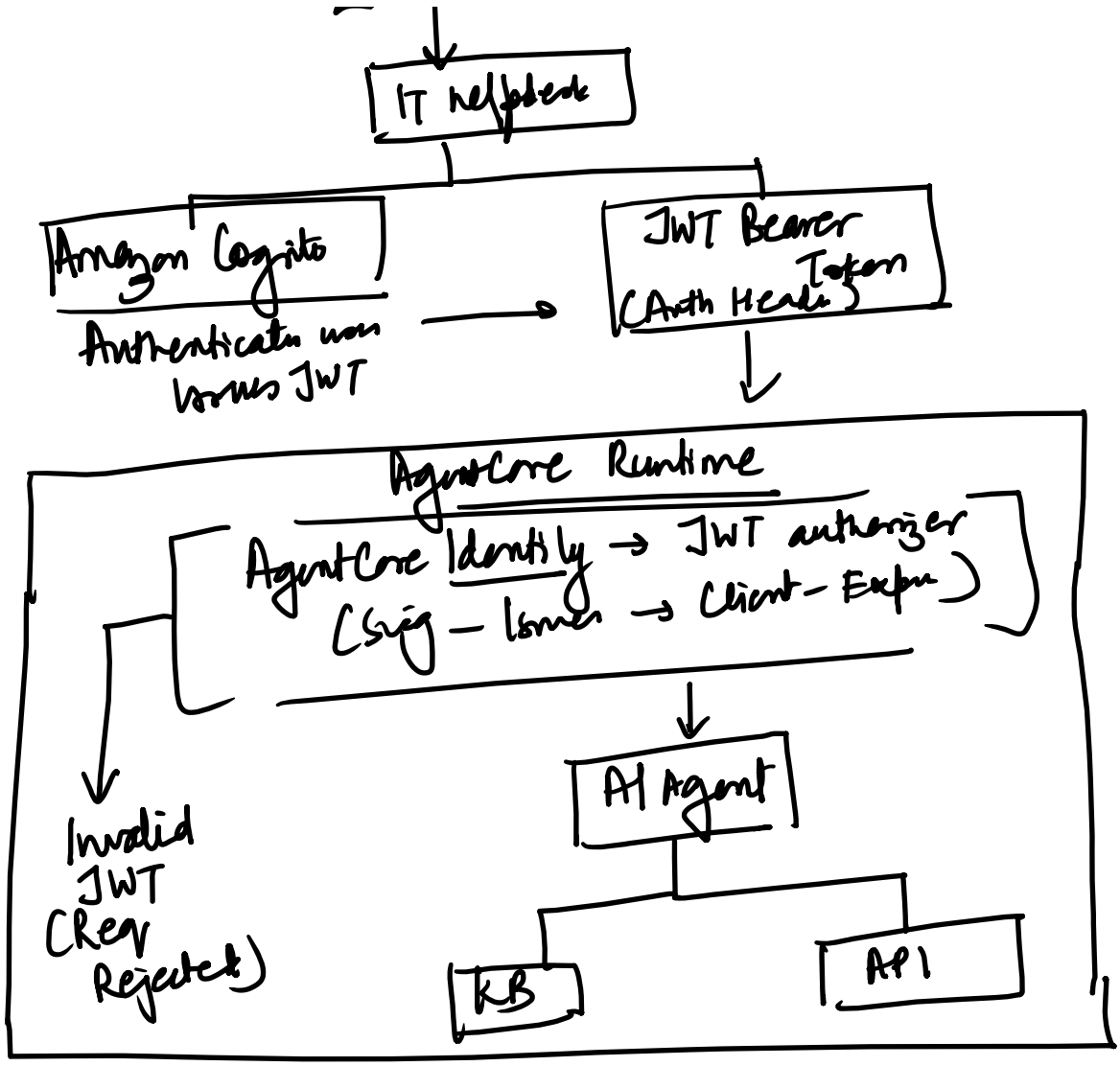
Demo: Monitor Agent Performance & Telemetry using AgentCore observability

(3) well-Architected Agentic AI Systems.

Demo: Secure AI Agent Access — AgentCore Identity

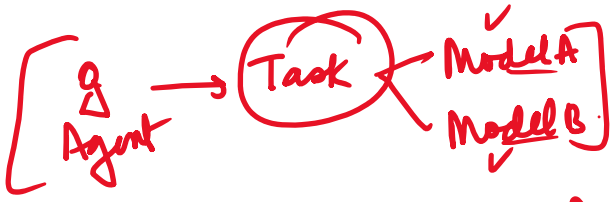
(1) Agent Dev → Strands Agent SDK
(2) Secure deployment → Runtime & Amazon Cognito JWT
(3) Authentication Testing.





Performance Metrics Collection & custom metrics

(1) Reasoning efficiency = How much work the agent performs to reach an answer?



(2) Context Utilization → Agent uses the information it retrieves effectively.

(3) Tool selection accuracy = Agent choose the Right Tool.

(4) Collaboration effectiveness = How well specialized Agents coordinate.

Demo: Monitor Agent Performance & Telemetry

Project folder

1. main.py → Agent (Primary) application code – Tools def & agent logic
2. Dockerfile → Container Build Config
3. invoke_agent.py → Helper script for testing agent invocations
4. redeploy.sh → Redeploying the agent to AgentCore Runtime
5. FinanceAgent/ → AgentCore CLI prog – AWS CDK

1. Test agent → Runtime
2. Generate Telemetry & Explore Runtime metrics.
3. Investigate errors using Cloudwatch log Analytics
4. Fix the agent code & Redeploy
5. Create Cloudwatch Alarms for proactive monitoring.

Well-Architected Principles for AgenticAI

1. Observability = what is the agent doing ?
   - ✓ Metrics = How the system ?
   - ✓ Logs → what event occured ?
   - ✓ Traces → The complete journey.
2. Controllability = To prevent, approve, interrupt, restrict actions.

(2) Controllability = to prevent, [illegible] actions.

(3) Veracity & Robustness

↓ Truthful & Accurate Results

↓ unexpected inputs & failures.

AgentCore Browser
- Session Recording & Replay

(4) Fairness ✓

(5) Transparency ✓

(6) Governance ✓

2LO & 3LO → AgentCore Identity

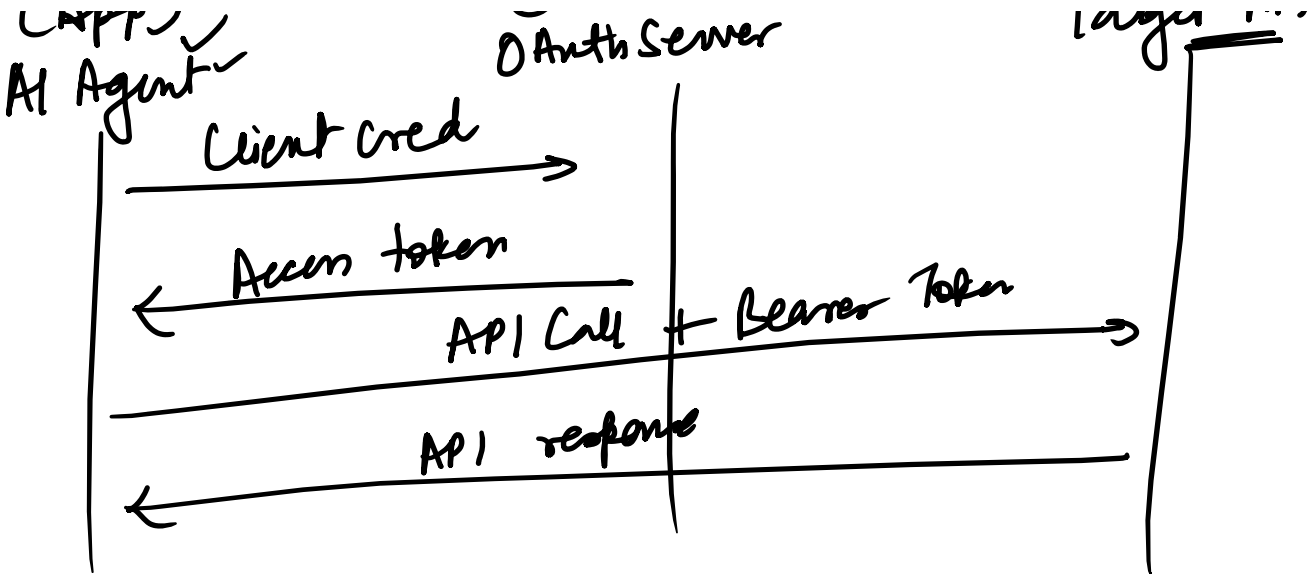
↳ OAuth patterns ⇒ AI Agents access the external Applicatn — Service Now, Salesforce, Google workspace, etc

2LO = Agent acts using own app identity without requiring individual user

3LO = Agent acts on behalf of a particular user

2Lo (Two-legged OAuth): Machine-to-Machine

(App) ✓ AI Agent ✓ → (store token) OAuth Server → (validate) Target API ✓


AI Agent
OAuth Server
Client Cred
Access token
API Call + Bearer Token
API response